Billing & Collections Microservice

System Design & API Implementation

Bill Heitzeg - March 16, 2026

Problem Overview

Design a microservice responsible for

- Manage policy premium billing
- Track payment attempts and outcomes
- Trigger payment reminders
- Retry failed payments
- Integrate with third-party payment processor

Service Responsibilities

Billing Service

- premium schedules
- delinquency determination
- reminder triggering

Payments Service

- payment execution
- payment attempt history
- retry logic
- third-party processor integration

Components

REST Endpoints

GET /policies/{policyId}/premium-schedule

GET /policies/delinquent

POST /payments

Queues

Payment

DB Tables

Payment

PaymentAttempt

PolicyBillingAccount

PremiumScheduleItem

ReminderLog

PaymentOutboxEvent

ReminderOutboxEvent

DelinquencyOutboxEvent

Workers

Payment Dispatcher

Payment Worker

Payment Result Publisher

Reminder Worker

Notification publisher

Delinquency Worker

External Systems

Third-Party Payment Processor

Notification Service

Database Schema

PolicyBillingAccount

policyId
delinquencyStatus
delinquentSince
updatedAt
outstandingAmount

PremiumScheduleItem

scheduleItemId
policyId
dueDate
amount
status

Payment

paymentId
policyId
scheduleItemId
status
attemptCount
nextAttemptAt
enqueued

PaymentOutboxEvent

eventId
paymentId
eventType
payload
published
publishedAt

ReminderLog

reminderId
scheduleItemId
reminderType
sentAt

PaymentAttempt

paymentAttemptId
paymentId
attemptNumber
result
failureReason
attemptedAt

ReminderOutboxEvent

eventId
scheduleItemId
eventType
payload
published
publishedAt

DelinquencyOutboxEvent

eventId
policyId
eventType
payload
published
publishedAt

- **Payment is the source of truth** for current state and retry lifecycle
- **PaymentAttempt provides a full audit trail** of all processing attempts
- **PremiumScheduleItem drives billing and reminders** for each policy
- **PolicyBillingAccount tracks delinquency status** at the policy level
- **Separate outbox tables ensure reliable async event publishing**
- **Published flags enable safe, idempotent event delivery**

Design Decisions

Asynchronous Payment Processing

Payments are placed on a **queue** and processed by workers to prevent API blocking and to isolate third-party payment latency.

Database as Source of Truth

All payment and billing state is persisted before processing to ensure **durability and recoverability**.

Dispatcher Pattern for Reliable Queueing

A **Payment Dispatcher** reads unenqueued payments from the database and publishes them to the queue, ensuring that no payment requests are lost if queue publishing fails.

Outbox Pattern for Event Delivery

Separate **Outbox tables** (PaymentOutboxEvent, ReminderOutboxEvent, DelinquencyOutboxEvent) guarantee reliable event publication without distributed transactions.

Idempotent Workers

Workers are designed to safely reprocess events to support **retry and failure recovery**.

Separation of Billing and Payment Responsibilities

Billing logic (premium schedules, reminders, delinquency status) is separated from payment execution to allow **different scaling and reliability requirements**.

Scheduled Workers for Policy State Evaluation

Background workers evaluate **reminders and delinquency status**, allowing the system to operate without blocking client requests.

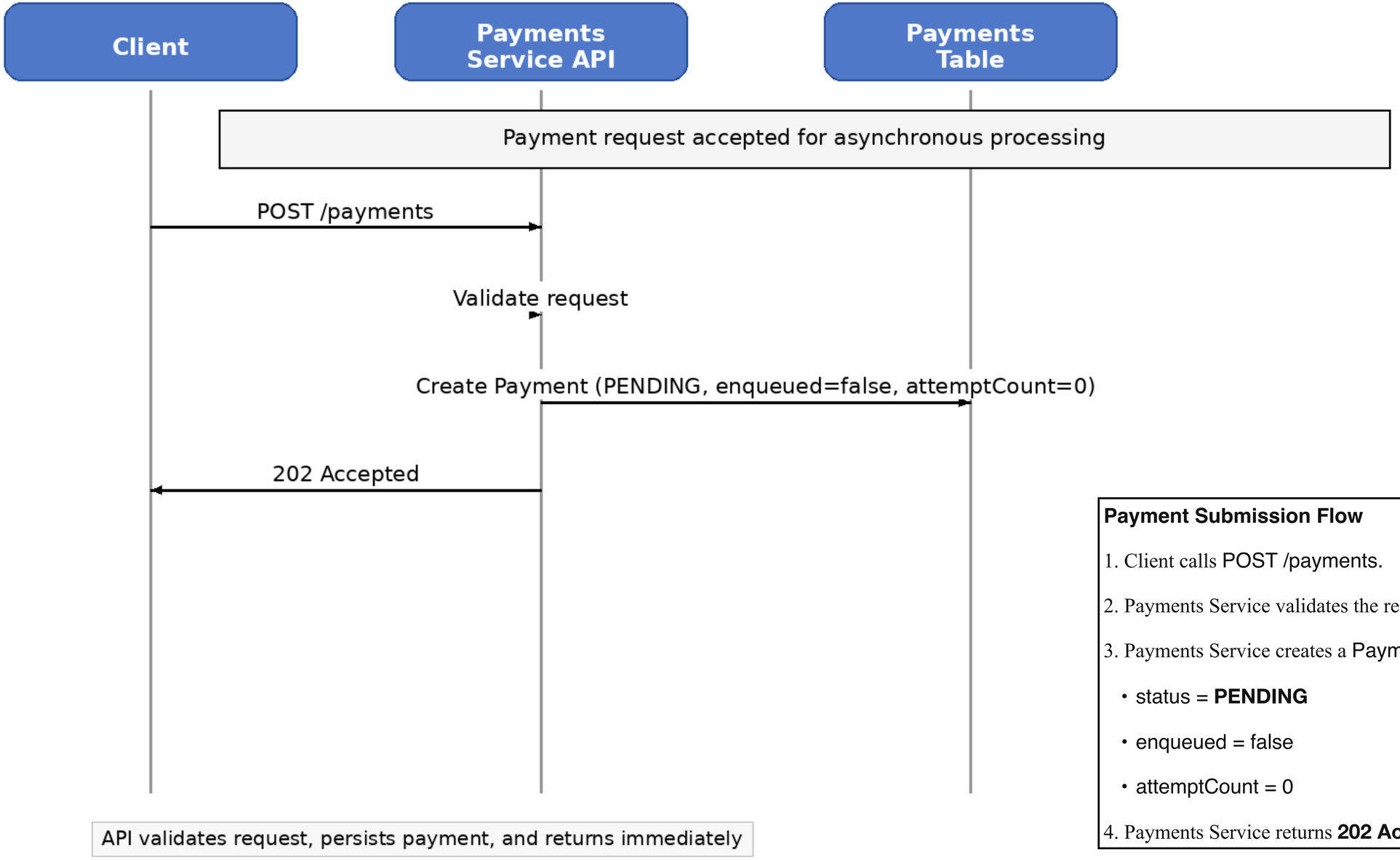
Extensible Event Model

Domain events such as **PaymentSucceeded** or **DelinquencyStatusChanged** allow external systems (collections, notifications, reporting) to react without tight coupling.

Payment Flows

- Payment Submission
- Payment Dispatch
- Payment Execution
- Payment Success
- Payment Failure/Retry

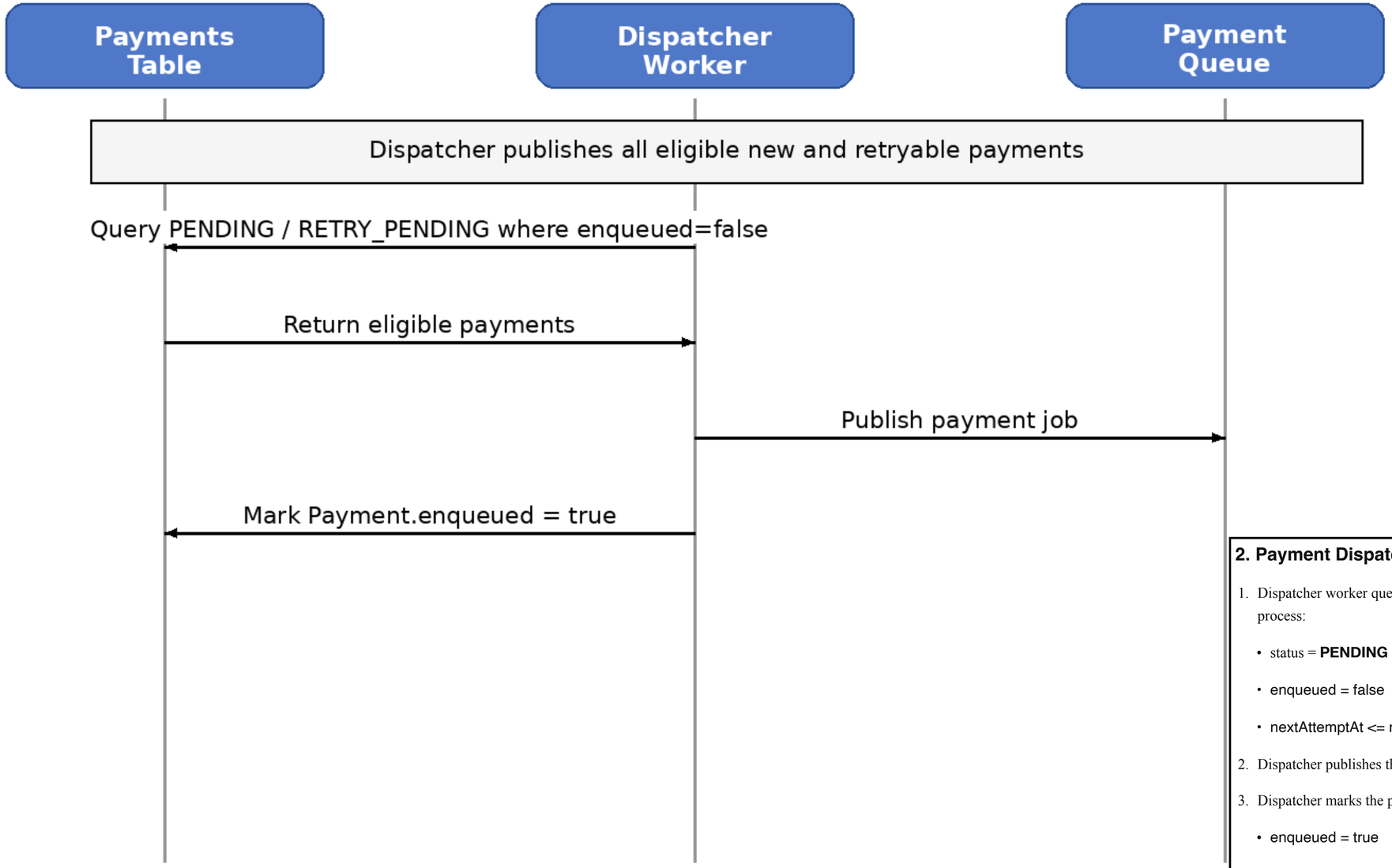
Payment Submission Flow



Payment Submission Flow

1. Client calls POST /payments.
2. Payments Service validates the request.
3. Payments Service creates a Payment record with:
   - status = **PENDING**
   - enqueued = false
   - attemptCount = 0
4. Payments Service returns **202 Accepted**.

Payment Dispatch Flow

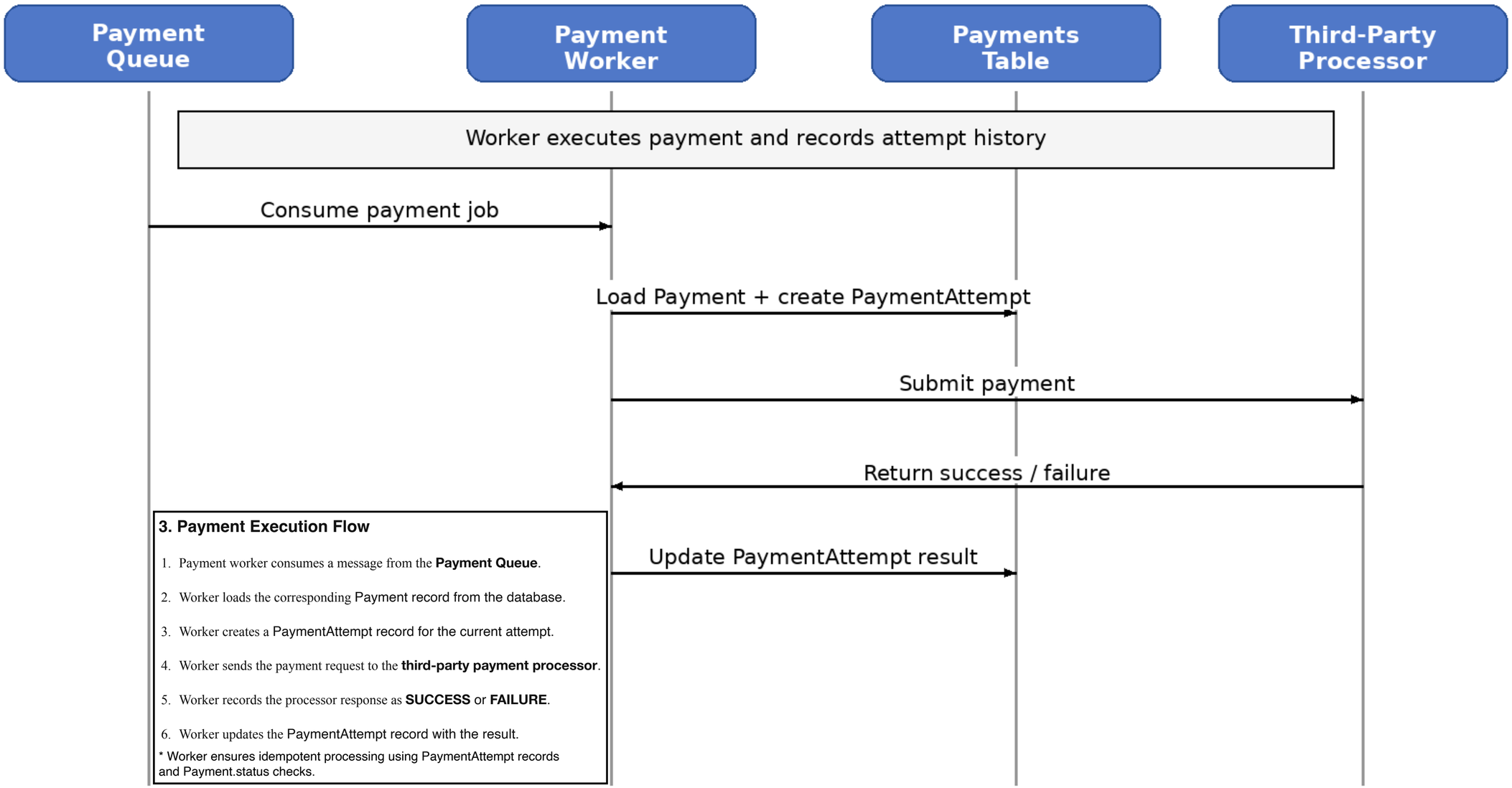


2. Payment Dispatch Flow

1. Dispatcher worker queries the Payments table for records that are ready to process:
   - status = **PENDING** or **RETRY_PENDING**
   - enqueued = false
   - nextAttemptAt <= now when applicable
2. Dispatcher publishes the payment to the **Payment Queue**.
3. Dispatcher marks the payment as:
   - enqueued = true

Payments DB is the source of truth; dispatcher handles all queue publication

Payment Execution Flow



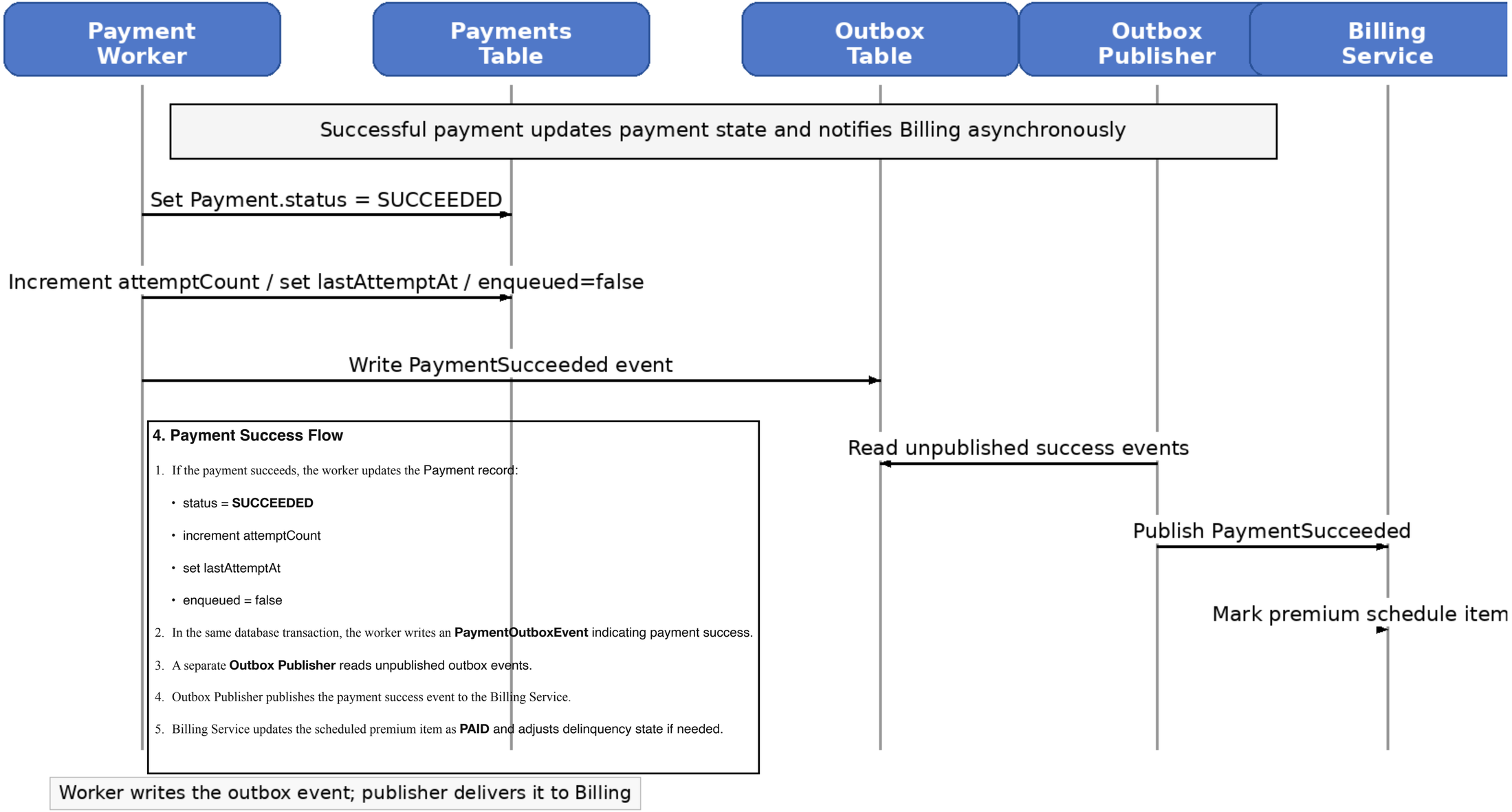
3. Payment Execution Flow

1. Payment worker consumes a message from the **Payment Queue**.
2. Worker loads the corresponding Payment record from the database.
3. Worker creates a PaymentAttempt record for the current attempt.
4. Worker sends the payment request to the **third-party payment processor**.
5. Worker records the processor response as **SUCCESS** or **FAILURE**.
6. Worker updates the PaymentAttempt record with the result.

* Worker ensures idempotent processing using PaymentAttempt records and Payment.status checks.

Payment worker owns processor interaction and attempt recording

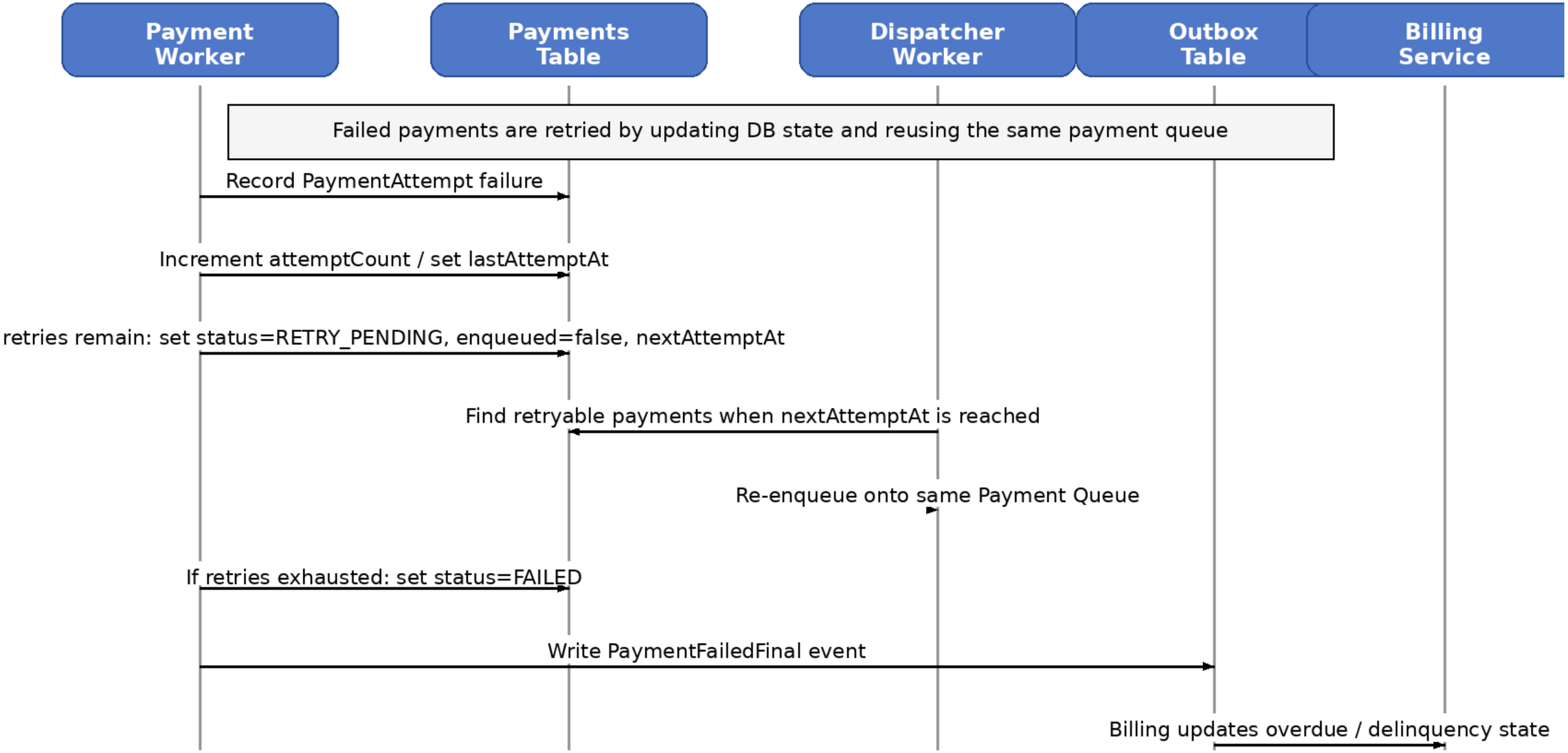
Payment Success Flow



4. Payment Success Flow

1. If the payment succeeds, the worker updates the Payment record:
   - status = **SUCCEEDED**
   - increment attemptCount
   - set lastAttemptAt
   - enqueued = false
2. In the same database transaction, the worker writes an **PaymentOutboxEvent** indicating payment success.
3. A separate **Outbox Publisher** reads unpublished outbox events.
4. Outbox Publisher publishes the payment success event to the Billing Service.
5. Billing Service updates the scheduled premium item as **PAID** and adjusts delinquency state if needed.

Payment Failure / Retry Flow



Retry state lives in the DB; dispatcher republishes retryable payments

5. Payment Failure / Retry Flow

1. If the payment fails, the worker updates the Payment record:
   - increment attemptCount
   - set lastAttemptAt
2. Worker checks retry policy.

If retries remain

3. Update the Payment record:
   - status = **RETRY_PENDING**
   - enqueued = false
   - set nextAttemptAt
4. Dispatcher will later re-enqueue the payment onto the same **Payment Queue**.

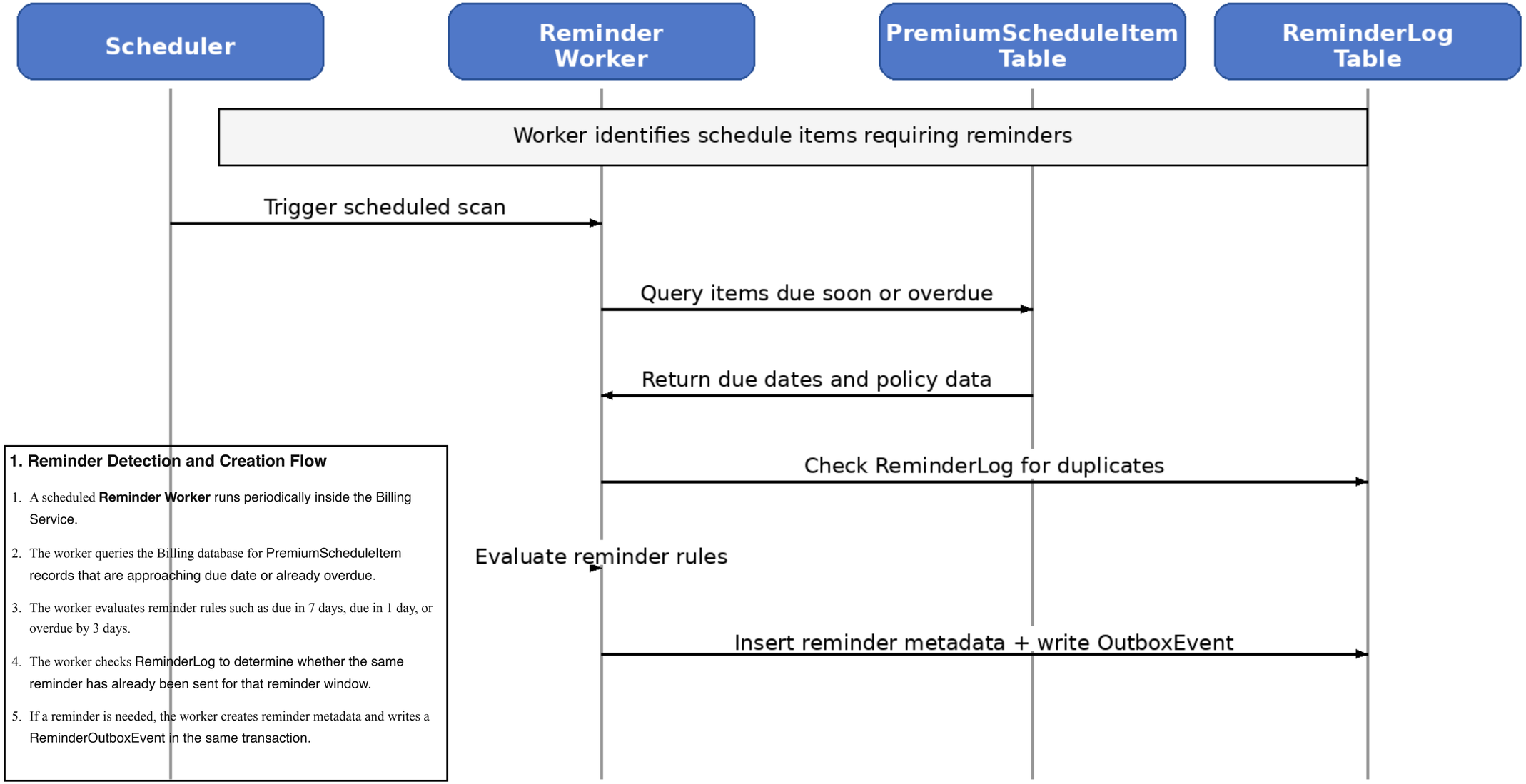
If retries are exhausted

5. Update the Payment record:
   - status = **FAILED**
   - enqueued = false
6. In the same database transaction, the worker writes an **OutboxEvent** indicating final payment failure.
7. A separate **Outbox Publisher** publishes the failure event to Billing.
8. Billing Service updates overdue/delinquency state as needed.

Payment reminder flows

- Reminder detection and creation flow
- Reminder delivery and outcome flow

Reminder Detection and Creation Flow

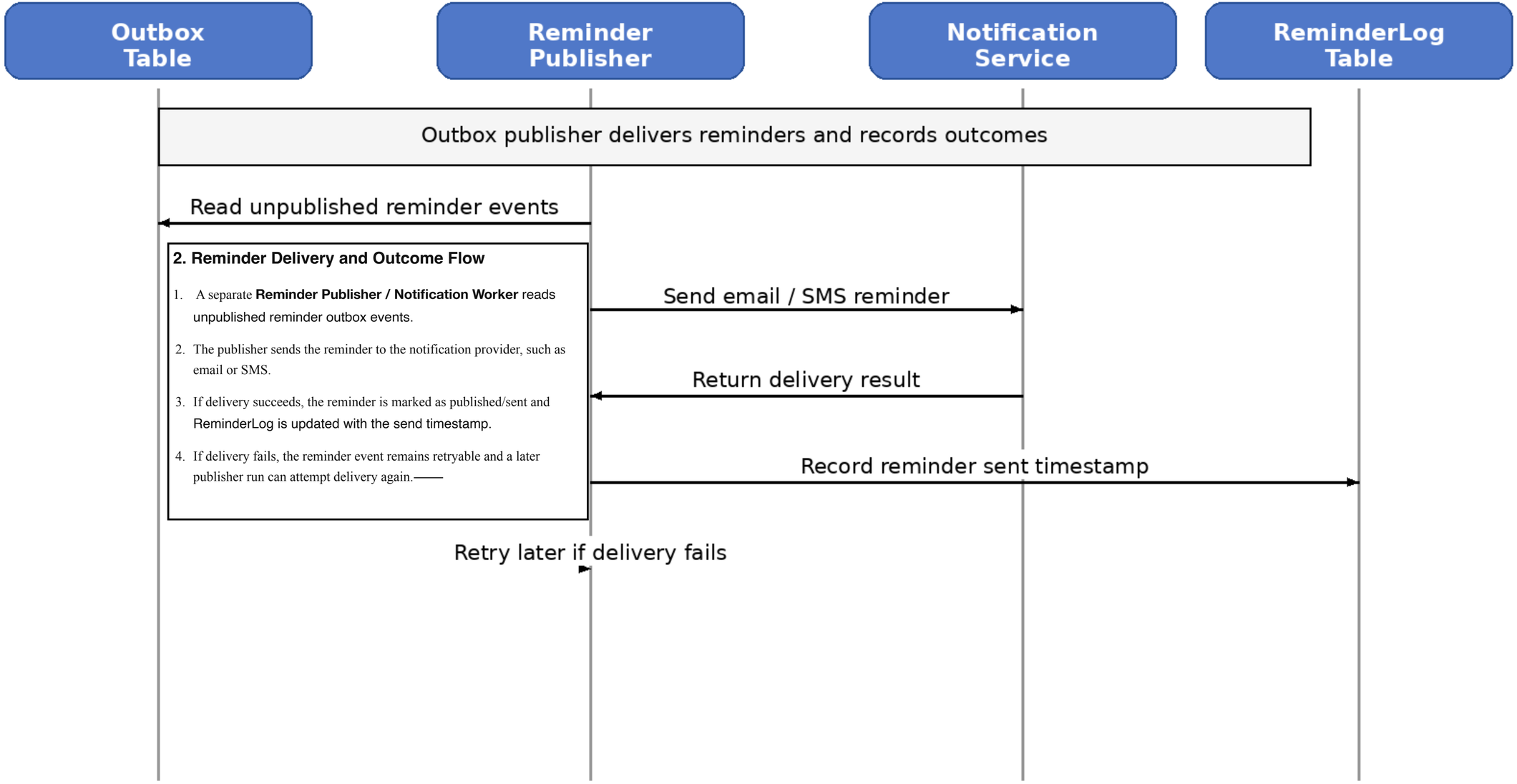


1. Reminder Detection and Creation Flow

1. A scheduled **Reminder Worker** runs periodically inside the Billing Service.
2. The worker queries the Billing database for PremiumScheduleItem records that are approaching due date or already overdue.
3. The worker evaluates reminder rules such as due in 7 days, due in 1 day, or overdue by 3 days.
4. The worker checks ReminderLog to determine whether the same reminder has already been sent for that reminder window.
5. If a reminder is needed, the worker creates reminder metadata and writes a ReminderOutboxEvent in the same transaction.

Detection, duplicate prevention, and reminder creation occur in one transaction

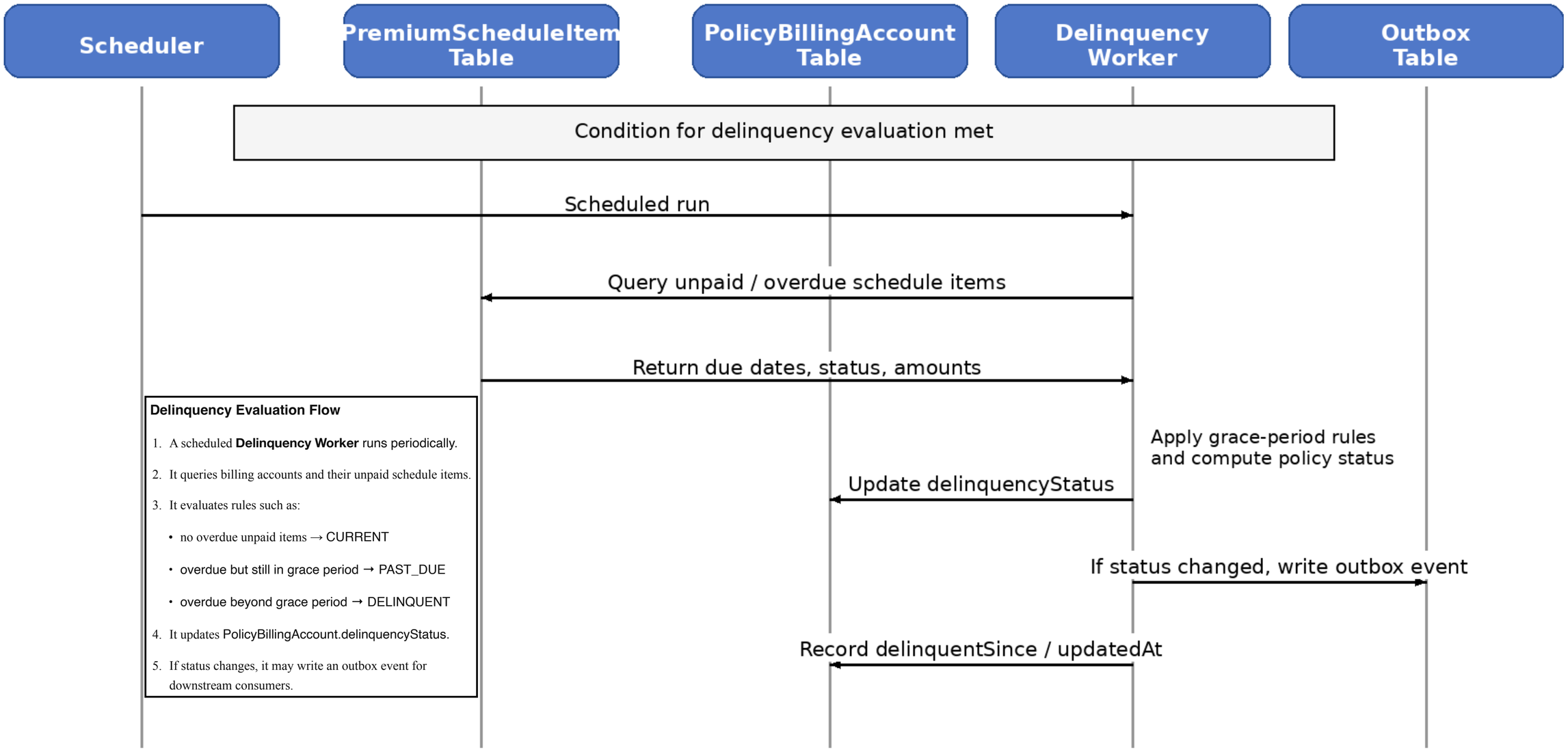
Reminder Delivery and Outcome Flow



Outbox ensures reliable delivery; ReminderLog stores final outcome

Delinquency Evaluation Flow

Delinquency Evaluation Flow



Delinquency Evaluation Flow

1. A scheduled **Delinquency Worker** runs periodically.
2. It queries billing accounts and their unpaid schedule items.
3. It evaluates rules such as:
   - no overdue unpaid items → CURRENT
   - overdue but still in grace period → PAST_DUE
   - overdue beyond grace period → DELINQUENT
4. It updates PolicyBillingAccount.delinquencyStatus.
5. If status changes, it may write an outbox event for downstream consumers.

CURRENT = no overdue unpaid items • PAST_DUE = overdue within grace period • DELINQUENT = overdue beyond grace period

Failure handling & Reliability

- Worker idempotency prevents duplicate processing
- Dispatcher ensures reliable queue publication
- Database is the source of truth for payment state
- Outbox pattern guarantees event delivery without distributed transactions
- Retries handled via persisted retry state and scheduled dispatch

Observability & Monitoring

- Structured logging with paymentId, policyId, attemptNumber
- Logs at key lifecycle events (enqueue, process, success, failure, retry)
- /actuator/health, /liveness, /readiness for system health
- /metrics and /prometheus for monitoring and alerting
- /info for build and version visibility

External Integration Resilience

Third Party Payment processor

- Retry with exponential backoff for transient failures
- Circuit breaker prevents cascading failures when processor is down
- Timeouts ensure worker threads are not blocked
- Idempotent processing prevents duplicate charges

Scalability & Production Considerations

- Horizontal scaling of workers for payment processing
- Partitioning/queuing by policyId to avoid contention
- Pagination for delinquent policies endpoint
- Database indexing on payment status and due dates
- Backpressure handling when third-party processor is slow

Tradeoffs & Simplifications

- In-memory queue used for simplicity → would use Kafka/SQS in production
- Single service implementation → design separates billing and payments logically
- Outbox modeled conceptually → would add dedicated publisher and transactional guarantees
- No authentication/authorization → assumed external gateway handles this